CSE UNDERGRADUATE · BANGLADESH UNIVERSITY

# Automate delivery. Operate with clarity. Build reliable systems.

I am MD SHOHAN, a Computer Science and Engineering undergraduate developing DevOps and Site Reliability Engineering foundations across systems, cloud infrastructure, automation, software delivery, observability, and security.

LEARNING-STAGE ENGINEERING PORTFOLIO

This is a structured technical knowledge map, not a claim of professional DevOps employment, production ownership, certification, or mastery across every vendor. No project gallery is published yet; practical work can be added when completed and verifiable.

LEVEL 01 / CORE ENGINEERING FOUNDATIONS

## Understand the substrate first.

SYSTEMS

### Linux & operating systems

Processes, signals, users, permissions, filesystems, packages, systemd, journald, SSH, namespaces, cgroups, and host troubleshooting.

NETWORKING

### Networking fundamentals

IP addressing and CIDR, subnetting, routing, TCP/UDP, ports, DNS, HTTP, TLS, firewalls, NAT, and network namespaces.

ARCHITECTURE

### Computer systems

CPU scheduling, memory, storage I/O, caching, concurrency, virtualization, hypervisors, and resource bottlenecks.

AUTOMATION

### Shell & programming

Bash for operating-system automation; Python for APIs, tests, structured workflows, and data transformation. Validate inputs and fail clearly.

CHANGE CONTROL

### Git & software lifecycle

Commits, branches, pull requests, reviews, tags, release notes, CI checks, and feedback from requirements through operations.

APPLICATIONS

### Software & APIs

HTTP methods/status codes, REST contracts, authentication, configuration, dependencies, timeouts, retries, idempotency, and health endpoints.

ENGINEERING DEFAULTS

**Traceable change** through Git and review. **Repeatable automation** through tested code. **Resilient systems** through health contracts and recovery paths. **Operational clarity** through useful telemetry and documentation.

LEVEL 02 / CORE DEVOPS ENGINEERING ECOSYSTEM

# From reviewed change to a running service.

Core DevOps joins source control, CI validation, build systems, artifact security, infrastructure provisioning, deployment, and runtime feedback.

| SOURCE | VALIDATE | BUILD | SECURE | PUBLISH | DEPLOY |
|---|---|---|---|---|---|
| Git · review | tests · lint | image · package | scan · SBOM | digest | IaC / GitOps |

CONTAINERS

### Docker & OCI containers

Layered, content-addressed images; namespaces/cgroups isolation; networks and volumes; multi-stage builds, minimal bases, non-root runtime, and Compose.

ORCHESTRATION

### Kubernetes fundamentals

Deployments, Pods, Services, ConfigMaps, Secrets, probes, requests/limits, Namespaces, Ingress/Gateway, and declarative reconciliation.

INFRASTRUCTURE

### Terraform & Ansible

Terraform declares cloud resources and tracks state; Ansible converges hosts through inventories, playbooks, roles, and idempotent modules.

DELIVERY

### CI/CD platforms

Jenkins, GitHub Actions, and GitLab CI runners, jobs, artifacts, reusable workflows, deployment environments, and scoped permissions.

CLOUD

### Cloud computing

AWS, Azure, and GCP compute, network, identity, storage, registries, managed Kubernetes, and observability under shared responsibility.

RELEASES

### Artifacts & release management

OCI and package registries, immutable digests, SBOMs, provenance, retention, rolling, blue/green, canary, and rollback practices.

## Infrastructure lifecycle

Declare configuration, initialize providers/backends, generate a plan, review policy and cost, approve, apply, verify outputs, inspect drift, and preserve secure state. A plan is a proposal, not proof of deployment.

> Use remote state with locking and encryption for team workflows. Keep credentials and environment-specific secrets out of source control. Prefer short-lived identity federation over static CI keys.

LEVEL 03 / ADVANCED DEVOPS & PLATFORM ENGINEERING

# Operate through declarative control loops.

Advanced engineering expands from individual jobs into cluster lifecycle, Git-based reconciliation, developer experience, distributed system boundaries, and failure-domain design.

### Kubernetes architecture

Control plane: API server, etcd, scheduler, controllers. Nodes: kubelet, container runtime, CNI, Pods, and CSI. Scheduling includes affinity, taints/tolerations, topology spread, priority, and capacity.

### Workload operations

Rolling strategies, startup/liveness/readiness probes, HPA, PDB, resource budgets, admission policy, RBAC, autoscaling, and cluster upgrades. HPA needs a working Metrics API; PDBs mainly constrain voluntary disruptions.

### Helm & Kustomize

Helm charts template reusable manifests with values; Kustomize bases and overlays patch resources per environment. Render and validate generated YAML before applying.

### GitOps: Argo CD & Flux

Controllers compare repository-defined desired state with the cluster and reconcile drift. Use protected source paths, digest-pinned images, and carefully scoped pruning/self-heal policies.

### Platform engineering

Internal developer platforms combine catalogs, templates, golden paths, documentation, ownership, and policy guardrails. Backstage can provide catalog and Scaffolder capabilities when integrated with an operating portal.

### Microservices & service mesh

Service boundaries add network failure, distributed data, and compatibility concerns. Meshes can centralize mTLS, identity, policy, and telemetry, but add operational cost.

### Resilience & disaster recovery

Reduce correlated failure across zones and dependencies. Define RTO/RPO, backups, restore drills, graceful degradation, redundancy, health-aware routing, and failover exercises.

### Serverless & hybrid cloud

Functions and managed services shift host operations but retain identity, quotas, retries, event delivery, observability, vendor limits, egress, and data-gravity concerns.

A resource manifest is not proof of runtime enforcement. Verify the target CNI for NetworkPolicy, the Metrics API for autoscaling, identity bindings, storage classes, admission controls, and real rollout/recovery behaviour.

LEVEL 04 / OBSERVABILITY, RELIABILITY & SECURITY

# Make service behaviour visible and defensible.

| WORKLOAD | METRICS | LOGS | TRACES | EXPLORE | RESPOND |
|---|---|---|---|---|---|
| instrument | Prometheus | events | OpenTelemetry | Grafana / backend | alert / runbook |

## Prometheus & metrics

Scrape time series, query PromQL, and evaluate recording/alert rules. Use counters, gauges, histograms, and bounded labels to prevent cardinality explosions.

## Grafana & logging

Build question-driven dashboards with deployment markers and links to logs/traces. Centralized logs need structured fields, access control, parsing, and retention.

## OpenTelemetry

Vendor-neutral APIs, SDKs, semantic conventions, and Collector pipelines generate/process/export metrics, logs, and spans. Context propagation connects distributed traces.

## SRE: SLIs, SLOs, SLAs

Define eligible good events, measurable indicators, target windows, and external commitments. Error budgets balance delivery risk with reliability investment.

## Incident management & RCA

Detect, declare, assign roles, mitigate, communicate, recover, and learn. Use runbooks, incident timelines, blameless postmortems, and owned corrective actions.

## DevSecOps & supply chain

SAST, SCA, secret scanning, IaC/container scanning, SBOMs, signing, provenance, policy checks, and patch response across the delivery lifecycle.

## IAM & secrets

Least privilege, scoped RBAC, OIDC federation, short-lived workload credentials, secret rotation, audit logs, and secure key management.

## Network security & governance

Segmentation, TLS, private endpoints, encryption, policy-as-code, asset inventory, audit evidence, and appropriate CIS/NIST/ISO-aligned controls.

## Backup & recovery

Define RPO/RTO, encrypt and isolate backups, test restoration, rehearse failover, and monitor the recovery process. Backup completion does not prove recoverability.

Useful operational views combine RED (rate, errors, duration), USE (utilization, saturation, errors), and Golden Signals (latency, traffic, errors, saturation). Alert on sustained user impact, with an owner, severity, threshold, and runbook.

LEVEL 05 / SUPPORTING ENGINEERING DISCIPLINES

# Understand the dependencies around the platform.

### APIs & databases

OpenAPI, auth, timeouts, retries, transactions, indexes, replication, connection pools, migrations, cache invalidation, and consistency.

### Traffic & web services

NGINX, HAProxy, Envoy, L4/L7 balancing, reverse proxies, forwarded headers, graceful draining, DNS, certificates, and TLS.

### Storage & messaging

Object/block/file storage, CSI, snapshots, queues, event delivery semantics, dead-letter queues, idempotent consumers, and lag monitoring.

### Documentation & collaboration

ADRs, runbooks, ownership, code review, onboarding, versioned diagrams, issue triage, and shared operational responsibility.

### FinOps & economics

Cost allocation, tagging, budgets, anomaly detection, right-sizing, egress, lifecycle, autoscaling, and performance/resilience trade-offs.

#### INTEGRATED ENGINEERING REFERENCE PATTERNS

Local observability: a Python API exposes liveness/readiness and Prometheus metrics; Docker Compose provides a local API + Prometheus + Grafana topology. Production requires secure ingress, durable telemetry, authentication, retention, alert routing, and an SLO policy.

Cloud and Kubernetes: a two-zone VPC separates public and private subnets; NAT and EKS can be cost-sensitive opt-ins. A hardened workload uses non-root execution, read-only filesystem, seccomp, probes, resource bounds, HPA, PDB, and narrowly scoped NetworkPolicy.

Delivery and platform: CI can test, build with Buildx, generate a CycloneDX SBOM, scan with Trivy, publish an immutable GHCR digest, and let Argo CD reconcile Helm releases. Backstage catalog/templates describe a service golden path but do not constitute a running portal.

#### EVIDENCE & SCOPE

The portfolio is intentionally project-free. The projects directory is empty. These pages explain technical patterns and learning areas; they do not claim a live cloud account, Kubernetes cluster, production observability platform, professional experience, certification, or completed project. Tools in the technology map are topics for study, not proficiency scores.

When practical work is ready to publish, it should include source, reproducible setup, architecture decisions, validation evidence, security boundaries, operational limitations, and an honest account of what was actually run.

**Contact** · mohammad.and.shohan@gmail.com · Dhaka, Bangladesh
**GitHub** · github.com/MD-SH0HAN · **Portfolio** · md-sh0han.github.io/PORTFOLIO/